

1 Summary

WebVOWL is an online ontology visualization tool which allows users to visualize ontologies as force-directed graphs. While this tool is in use by both researchers and practitioners, it suffers from significant performance issues which severely limits the size of ontologies able to be visualized. Furthermore, its search functionality prevents users from locating any element which is not currently rendered on screen.

In this paper, we present our efforts to reduce loading time and memory usage, both of which were done by profiling the tool at runtime and performing time complexity analysis of key functions and methods in the code. Our analysis revealed that many of these functions suffered from poor asymptotic complexities, which in most cases could be eliminated by utilizing datastructures such as sets or maps. We conducted benchmarks, comparing the old version to ours, using real data of different sizes, each size increasing node and edge count by two orders of magnitude from the previous. The performance improvements to loading times have proven to be quite significant in these benchmarks; an ontology which before took 631.7 seconds to load can now be loaded in 1.23 seconds, i.e. 514 times faster.

However, after having improved performance considerably, we found that memory usage became the limiting factor during our preliminary benchmarks; we were unable to load the YAGO knowledge graph, with 166425 edges and 132882 nodes, on a laptop with 32 GB of RAM due to the browser running out of memory. After analyzing WebVOWL’s memory usage, we concluded that refactoring the entire codebase from ECMAScript5 to ECMAScript6 was the best course of action. This reduced memory usage by up to 57%, allowing us to load YAGO on our benchmark laptop with 8 GB of RAM.

Additionally, we present an extended search functionality. It allows the user to search for any element in the graph and if the searched element is not currently rendered on screen, a subgraph is created such that the searched element and its nearby connected elements are shown, replacing the previously rendered graph. Using the *Reset* button, the user is then able to return to the big picture view of the graph.

Finally, we want to highlight that this paper includes some elements from our demo paper accepted at The Extended Semantic Web Conference 2025 and published by Springer Nature.

Making WebVOWL Great Again: Improving Performance and Search

Frederik Schnack Pretzmann, Kristian Emil Weinreich Nielsen, Magnus Lindholm Rennekamp, and Nikolas Frederik Niemi

Aalborg University, Aalborg Øst, Denmark

Abstract. WebVOWL is an ontology visualization tool which renders ontologies as force-directed graphs that can be explored visually by the user. However, it suffers from poor performance which severely limits the size of ontologies able to be visualized. In this paper, we present our efforts to reduce loading time and memory usage, both of which were done by profiling the tool at runtime and performing complexity analysis of key functions and methods in the code. Furthermore, we extend the search functionality of WebVOWL with a new feature. This enables it to show searched, non-rendered elements as a subgraph centered on the searched element and its nearby connected elements. We conduct benchmarks of the improvements using real data of different sizes, each size increasing the node and edge count by two orders of magnitude from the previous. In these benchmarks, we see drastic improvements to load times as well as significant improvements to memory usage.

Keywords: Ontology · Web · Visualization

Preface

This paper is part of our bachelor project. It includes some elements from our demo paper accepted at The Extended Semantic Web Conference 2025 and published by Springer Nature [24].

1 Introduction

Since the emergence of the Semantic Web in 2001, interest in using ontologies as a method of describing concepts in a machine-readable representation has risen [2]. Ontologies can provide additional context to information on the internet, which can then be used by software agents to perform complex queries on large datasets [13].

To standardize ontology development, The Web Ontology Language (OWL) was established as a W3C recommendation in 2004 [26]. OWL’s syntax builds on RDF/XML, ensuring interoperability across systems [14]. Its successor, OWL 2, introduced enhanced expressivity and became a standard in 2009 [11]. Despite

OWL’s adoption by both experts and non-specialists, the complexity of large-scale ontologies poses challenges for users. Visualization tools like the Visual Notation for OWL Ontologies (VOWL) address this issue by translating abstract ontology structures into intuitive graphical representations [15].

WebVOWL and ProtégéVOWL are prominent implementations of the VOWL specification. These tools render ontologies as interactive, force-directed graphs, mapping classes, properties, and hierarchies to visual elements. While ProtégéVOWL integrates with the widely-used Protégé ontology editor, WebVOWL offers a web-based platform accessible via standard browsers [18,19]. However, these implementations struggle with large ontologies, as the original VOWL authors note that users found it difficult to find relevant information, and performance degrades when loading and rendering large ontologies [15].

In this paper, we present an updated version of WebVOWL, with improved performance and extended search functionality, facilitating WebVOWL’s use with large ontologies while maintaining the original user experience.

The structure of the rest of this paper is as follows: We give an overview of the current state of the ontology visualization field after the original release of WebVOWL in Section 2. In Section 3, we introduce the relevant notation and theory for our implemented changes. Section 4 lists the limitations we found in the original implementation of WebVOWL, and how these limitations were addressed in our implementation. Experimental results comparing our changes with the original WebVOWL in terms of time and memory used are given in Section 5. In Section 6 we discuss our findings and their implications, before the paper is concluded in Section 7.

2 Related Work

The original authors of VOWL noted that most existing ontology visualization tools had been designed for ontology experts, while VOWL aims to give an intuitive understanding, useful for both experts and users inexperienced with using ontologies [15]. This is supported by the visualization mantra of “*overview first, zoom and filter, then details-on-demand*” cited in the original paper, giving users an impression of the entire ontology, before focusing in on specific parts [25]. Some alternative ontology visualization tools used are extensions to the Protégé ontology editor.

Protégé is a knowledge representation tool developed by researchers at the Stanford Center for Biomedical Informatics Research [21]. It was originally developed to generalize OPAL, a tool for building and maintaining cancer chemotherapy knowledge bases [22]. Protégé allowed developers to create domain models, or modify existing models like OPAL. Since its creation in 1989, Protégé has continuously received updates, and at the time of writing, the Protégé website displays that at least 366,084 people have used the tool [3].

Protégé remains in use for creating, modifying, and visualizing OWL ontologies, and different plugins have been developed for visualizations, including

ProtégéVOWL for visualizing ontologies in the VOWL notation [19]. These plugins are limited to the desktop version of Protégé, which is platform independent.

In 2018, Dudáš et al. published a survey of state-of-the-art ontology visualization methods and tools [8]. Dudáš et al. evaluated 33 different visualization tools, including WebVOWL, comparing them based on their visualization methods, interaction techniques and supported ontology structs. Additionally, performance was measured by whether the tool was able to load and display a large ontology (the SUMO ontology [23]) and only 5 tools passed this requirement: Ontodia, OntoGraf, the Protégé Entity Browser, TGViz, and TopBraid. Of these tools, Ontodia, OntoGraf and TopBraid only display part of the loaded ontology manually selected by users, while the Protégé Entity Browser displays entities in an indented list. TGViz, a plugin for Protégé, displays ontologies as force-directed graphs similar to WebVOWL. However, it has not been updated since 2005, and is incompatible with Protégé versions newer than 3.1.1 [1].

3 Preliminaries

In this section we present the relevant theory and introduce the notation used for describing our changes to WebVOWL.

3.1 Ontologies as Graphs

An ontology is a formal, explicit specification of concepts, defining classes, properties, and axioms that model a knowledge domain.

The VOWL notation visualizes ontologies by representing them as graphs. In this context, nodes denote classes, while edges denote properties and relationships between them. Table 1 shows some of the OWL constructs as represented in VOWL notation. While VOWL represents more constructs than included in table 1, when describing the changes presented in this paper these can be seen as different labels on the same constructs, i.e. specialized nodes and edges.

Table 1. Basic OWL constructs in VOWL [15].

Primitive Application	
	classes
	properties
	property directions

As WebVOWL represent classes and properties as nodes and edges in a graph, we can take advantage of graph properties in our improvements, and for this purpose we give a formal description of a graph in VOWL. A VOWL graph can

be described as a labeled, directed multi-graph permitting loops, for which we use the following notation:

A labeled graph $G = (V, E, \phi, L)$ (1)

A node $v \in V$ (2)

An edge $e \in E$ (3)

An incidence function¹ $\phi : E \rightarrow \{(v_1, v_2) \mid (v_1, v_2) \in V^2\}$ (4)

A label for an edge or node $L : V \cup E \rightarrow Label$ (5)

Degree of collapsing $d \in \mathbb{N}_0$ (6)

Degree of a node $deg(v) = \sum_{e \in E} \begin{cases} 1 & \phi(e) = (v, v') \\ 1 & \phi(e) = (v', v) \end{cases}$ (7)

Number of edges in shortest path² $dist : V^2 \rightarrow \mathbb{N}_0$ (8)

Here, classes are represented as nodes, properties as edges, and axioms as triples $(L(v_1), L(e), L(v_2))$ where $\phi(e) = (v_1, v_2)$, for (subject, predicate, object). The label of an edge or node includes any information used to differentiate elements in the visualization. As illustrated in figure 1, this includes information related to the color, shape, and name of nodes and edges.

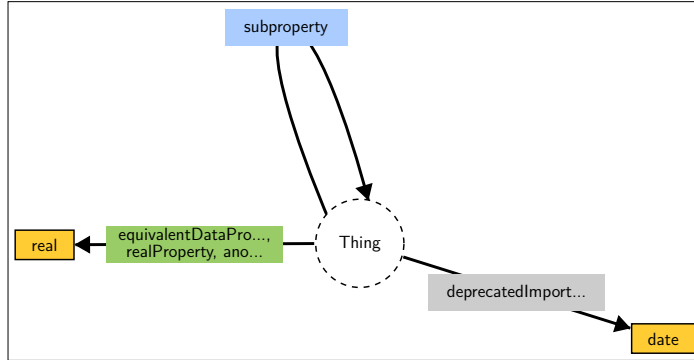


Fig. 1. Example of entities and relations taken from the benchmark ontology OntoViBe [12].

To limit the amount of information presented, WebVOWL includes filtering options, which remove nodes and edges based on the following user-specified options:

- Datatype properties
- Object properties

¹ Mapping an edge e to its tail v_1 and head v_2 by $\phi(e) = (v_1, v_2)$

² The distance from a node to itself is 0.

- Solitary subclasses
- Class disjointness
- Set operators
- Degree of collapsing

These filters are either enabled or disabled, with the exception of *degree of collapsing*, which creates a subgraph G' from G based on a specified degree d :

$$\begin{aligned} G' &= (V', E', \phi, L), \text{ where} \\ V' &= \{v \in V \mid \deg(v) \geq d\}, \\ E' &= \{e \in E \mid \phi(e) \in \{(v_1, v_2) \mid v_1, v_2 \in V'\}\} \end{aligned} \quad (9)$$

The search functionality of WebVOWL currently works by highlighting elements in the subgraph G' created by applying all enabled filters. In order to allow the search function to display nodes outside of this subgraph, without rendering the entire graph, we propose using breadth-first search at a fixed depth $k \in \mathbb{N}^+$, starting at the searched node [7, p. 556]. This then results in a subgraph G_s from G with a searched node v' :

$$\begin{aligned} G_s &= (V_s, E_s, \phi, L), \text{ where} \\ V_s &= \{v \in V \mid \text{dist}(v', v) \leq k\} \\ E_s &= \{e \in E \mid \phi(e) \in \{(v_1, v_2) \mid v_1, v_2 \in V_s\}\} \end{aligned} \quad (10)$$

3.2 Asymptotic Notation

We describe our performance improvements in the WebVOWL implementation using asymptotic notation where applicable. Generally, we use the worst case, describing the upper bound of a computation. However, given that WebVOWL is intended to be used with real ontology data, the worst case may not match general use, e.g. a graph where every node is connected to every other node does not match any real ontologies used in testing. Because of this, best-case runtime is sometimes included as a lower bound when a change lead to improvements in testing, but not in worst-case runtime. Formally, improvements are measured using the following definitions [7, pp. 53-56]:

Worst case:

$$\begin{aligned} O(g(n)) &= \{f(n) \mid \text{there exist positive constants } c \text{ and } n_0 \text{ such that} \\ &\quad 0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0\} \end{aligned} \quad (11)$$

Best case:

$$\begin{aligned} \Omega(g(n)) &= \{f(n) \mid \text{there exist positive constants } c \text{ and } n_0 \text{ such that} \\ &\quad 0 \leq c \cdot g(n) \leq f(n) \text{ for all } n \geq n_0\} \end{aligned} \quad (12)$$

3.3 Trie

For efficient lookup of strings that match an inputted search term, we propose using a specialized search tree known as a trie (sometimes called digital search

tree). A trie is generated from a set of words W , such that traversing the tree from a word w_s leads to a subtree containing all words in W with w_s as a prefix [17, pp. 492-494]. Using a trie provides efficient search at the cost of additional memory, as the number of nodes in the tree $|V_T|$ grows with the size and number of words.

A standard trie allows $O(|w_s|)$ search and insertion, where $|w_s|$ is the length of the input string [17, pp. 492-494]. However, in order to return all nodes with the searched prefix, we traverse the remaining subtree. In the worst case, where all words in W have a common prefix w_s , this traverses $|V_T| - |w_s|$ additional trie-nodes, leading to a time complexity of $O(|V_T|)$. In practice, however, the amount of additional trie-nodes traversed is limited by the set of words $W_g \subseteq W$ shown in the GUI³. Thus, if the number of words contained in V_T , which share w_s , is greater than $|W_g|$, we traverse enough nodes to get $|W_g|$ words. Formally, we traverse at most $\tau = |w_s| + \sum_{i=1}^{|W_g|} |w_i| - |w_s|$ nodes, where $|w_i|$ is the length of the i 'th word in V_T closest to w_s . This yields an average-case time complexity of $O(\tau)$, which allows the trie to scale much better with larger ontologies.

3.4 JavaScript Memory Management

JavaScript is a garbage-collected language, meaning that the engine automatically allocates memory for variables and objects, and reclaims memory when no longer needed. Typically, primitives and function call frames are stored on the call stack, while objects and arrays are stored on the heap. The call stack can automatically clear memory that is no longer in use, while heap memory must be periodically freed by the garbage collector. The implementation of garbage-collection depends on the engine used, however, all modern JavaScript engines use implementations of the mark-and-sweep algorithm to determine when references are no longer needed [6].

Mark-and-sweep starts from a set of “roots”, including the global object and currently active function scopes. The algorithm then repeatedly traces all reachable objects from the previously visited objects, “marking” any new objects visited, and finishes by “sweeping” any unmarked objects [16].

For variable declarations, whether a variable can be freed can be determined by whether it is still in scope, as JavaScript uses *var* which is function-scoped, and *let* and *const*, which are block-scoped. Block-scoped variables can be freed as soon as the block exits, which is one reason that using *let* and *const* has become the standard over *var* in many style guides, including MDN’s [5].

4 Introducing a New and Improved WebVOWL

Here, we describe our efforts to reduce loading time and memory usage combined with an analysis of the performance gains. We do this by first describing how we approached finding the various problems in the codebase, followed by what

³ Its size is hard-coded to 8.

each of these problems were. Having done this, we then describe our proposed solution to each issue, how we implemented them, and how the change affects performance asymptotically. Following the explanation of performance-related changes, we then explain how we have extended the search functionality with a new feature allowing users to search and display filtered nodes. Finally, we briefly discuss how the overarching plan of the project had changed over time and the considerations that affected these decisions.

4.1 Analysis of WebVOWL Performance and Functionality

We performed performance analysis and run-time profiling of WebVOWL under the stress of a large ontology, namely ENVO⁴, with 7,038 nodes and 12,387 edges. Profiling was done using the Firefox profiler on a Windows 11 Home HP ENVY Laptop 13 with 8 GB of RAM and a Intel(R) Core(TM) i5-10210U CPU running Firefox v136.0.1 (64-bit), using WebVOWL v1.1.7⁵. While profiling, we found that several methods and function calls were taking up the vast majority of compute time in loading ontologies, indicating a set of potential bottlenecks for load performance. In the following, we analyze the root causes of each bottleneck. Unless mentioned otherwise, n represents the number of edges and m the number of nodes.

Edge and property creation Creation of edges (in the code called "links") entails numerous calls to `countAndSetLayers` and `countAndSetLoops`. Both functions iterate over each link and compare it to every other link to find whether there was more than one link between the two nodes it connected, and whether it connected a node to itself, resulting in $O(n^2)$ worst-case time complexity. Creating the initial link objects from the loaded file performs an $O(n)$ calculation n times resulting again in $O(n^2)$ worst-case time complexity.

Additionally, there is also an issue when creating the property structure, i.e., when properties are connected with their sub- and super-properties. As this is an involved process, we describe it briefly: a for-loop iterates through each raw property, and at one point need to hide some properties that have been merged. To do so, it checks whether the property was rerouted, and whether there is another equal property, which in itself is a check comparing the property to every other property. Thus, its worst-case time complexity is $O(n^2)$ as well.

Storing edges on nodes In the original WebVOWL implementation, the `storeLinksOnNodes` is called after creating links in the context of loading or refreshing, as well as inside the filter function. It takes an array of link objects and an array of node objects, and stores each link on the two nodes it is connected to. The function uses a nested for-loop iterating over each node in the outer layer and each link in the inner layer, checking if the domain or range of

⁴ <https://bioportal.bioontology.org/ontologies/ENVO>, accessed 13/05/2025

⁵ <https://github.com/VisualDataWeb/WebVOWL>, accessed 12/03/2025

the link was a match with the node. It would then save each matching link on the current outer loop node in the inner loop, resulting in $O(n \cdot m)$ worst-case time complexity.

Combining classes and properties WebVOWL parses data from a VOWL-compliant JSON file, which can be generated using the OWL2VOWL converter⁶, or uploaded directly from the GUI. This JSON file includes attributes for classes and properties, which define information such as a class being external, or a property being an object or a datatype. During parsing, these attributes are connected to the JavaScript objects representing classes and properties. Previously, this was done by comparing all classes to all class attributes, and all properties to all property attributes. Since these are one-to-one ratios with one class attribute for every class and vice versa, the resulting worst-case time complexity can be written as $O(\beta^2)$, where β is the largest amount of either classes or properties.

Finding the starting degree of collapsing When loading a sufficiently large ontology, it is key that the degree of collapsing set on loading is not too low, as this could lead to rendering an overwhelming amount of nodes that could both clutter the screen and cause visual lag. Thus, we find the lowest degree of collapsing which would result in rendering less than a set limit of nodes. Previously, this was done by starting from degree $i = 0$, applying the degree filter on the data, then checking the length, l_r , of the result and comparing this value to the maximum degree α . If $l_r > \alpha$, it would then check for the $i + 1$ 'th degree. On large ontologies, the optimal degree of collapsing can be in the hundreds, meaning that this calculation could be performed many times. Asymptotically, it has a worst-case time complexity of $O(\alpha \cdot (n + m))$.

Memory usage The original WebVOWL was primarily built back in late 2014 and early 2015, many features of modern JavaScript were not available to the original developers. Particularly the features of ECMAScript6 (ES6) such as classes, promises, maps, and sets. While all of these are used in WebVOWL, the majority of the codebase was built before their introduction [9]. This means that classes and inheritance structures for all objects follow the old ECMAScript5 (ES5) way of function prototype-based classes. The general notion of prototypes in ES5 is to reference the functions in the prototypes when instantiating classes to avoid allocating memory for the functions in every instance. The original WebVOWL codebase does use this approach, however, when profiling memory usage we still found a lot of space allocated for functions. This seems to be originating from multiple places in the codebase that does not use the prototype approach, instead adding functions onto objects at runtime.

Another issue we found is that the string created by the JSON parser reading the ontology file may take up a significant portion of memory, and doubly so

⁶ <https://github.com/VisualDataWeb/OWL2VOWL>, accessed 7/5/2025

as strings are encoded using UTF-16, which, in JavaScript, is stored in 16 bit integers [10]. The issue here is particularly that the JSON string is held in a long-lived scope and is then used in parsing which is called in the loading process. This means that the JSON string is kept in memory until loading is completed. For instance, the YAGO knowledge graph, shown in our benchmarks in Section 5, with 166425 edges and 132882 nodes, is represented by a 1.5 GB JSON string. Additionally, each loaded ontology was also intentionally cached⁷.

In practice, the only limiting factor for the original WebVOWL has been load time, as the time it would take to load enough data for memory usage to become an issue rendered the problem irrelevant. However, with new improvements to load time, we are now able to load ontologies of a size where memory usage becomes significant. During our preliminary benchmarks, we were unable to load the YAGO knowledge graph on a laptop with 32 GB of RAM due to the browser running out of memory.

Garbage collection The original WebVOWL only uses *var* variable declarations rather than *let* declarations, as *let* is another feature introduced in ES6. The result of this is that the memory has to be cleaned up once the garbage collector decides that these are no longer of use, as *var* declarations are function-scoped. If these were instead block-scoped *const* and *let* declarations where applicable, the JavaScript engine would know implicitly that said declarations could be released from memory when they left scope. Outside of using more memory than needed, these function-scoped declarations also lead to slower load times, as the garbage collector has to periodically compute when to deallocate this memory at runtime.

Merging equivalent properties In the original WebVOWL, the merging of properties with equivalents was done by iterating over each property, finding its equivalents, making a merged node, and then hiding the nodes of the things that were merged. However, the process of hiding merged nodes was done by iterating over each equivalent property and comparing it to the domain or range of every other property. This yields a time complexity of $O(n^2 \cdot \epsilon)$, where ϵ is the set of equivalents of each property⁸. This has an unlikely worst case where all properties are equivalent with all other properties, i.e. $\epsilon = n$, resulting in $O(n^3)$.

Filtering solitary subclasses One of the optional filters that can be applied to an ontology is to filter out subclasses that are only connected to their superclass, as the original VOWL authors identified that these can be filtered out without affecting the overall view of the ontology [15]. In the original WebVOWL, this process involves iterating over all properties to identify subclass relationships. To determine which subclasses can then be removed, they need to fulfill both of the following conditions:

⁷ Saved in a global variable that did not expire during the session

⁸ Including the property itself.

1. All connected properties are subclass properties.
2. The subclass does not inherit from multiple superclasses.

In the worst case where every property is a subclass, checking these conditions results in $O(n \cdot (n + m))$ time complexity, as all properties are checked for each subclass, to find connected properties.

Searching for graph elements Searching for an element in the ontology allows a user to quickly find what they are looking for without having to visually search for it among all rendered elements. The original WebVOWL implemented search by iterating over an array of strings $S[1 \dots k]$, containing all element names in the graph, and for each string $S_i[1 \dots f]$, checks if the search term $P[1 \dots t]$ is a substring of S_i using the JavaScript `indexOf` method. This method is, as of the 2024 ECMAScript standard [10], implemented using the naive string-matching algorithm described by Cormen et. al [7, p. 960]. This results in $O(k \cdot t(f - t + 1))$ worst-case time complexity.

4.2 Performance Improvements

Here, we describe the performance improvements implemented in the new version. For each improvement, we describe the implemented change and its expected benefit. In Section 5, we present the effect of applying each fix on the asymptotic time complexity in addition to the actual load time using real data. With these fixes, WebVOWL turns out to be performant in both the initial load and when users change the degree of collapsing. Additional runtime improvements are limited by the need to visually render many nodes at a time. Such large-scale rendering is required if the user lowers the collapse degree showing over a thousand nodes at once. After performance improvements, we also describe our extension to the search functionality of WebVOWL.

CountAndSetLayers To determine if two nodes have one or multiple edges between them, the original implementation (algorithm 1) uses a nested for-loop on an array containing all edges. Here, we use *edgeOuter* for the current edge of the outer loop, and *edgeInner* for the current edge of the inner loop. Whenever *edgeInner* has the same head and tail as *edgeOuter*, *edgeInner* is added to an array, and the completed array is added to *edgeOuter* as an attribute. This array is never accessed, and is only used to check if its length is 1. The updated version (algorithm 2) uses two sequential for-loops, the first adding values to a hashmap, with node-id keys and values counting the number of insertions with the given key. The second loop then reads the hashmap, adding the number of neighbor edges to each edge. From this change, time complexity is improved from $O(n^2)$ to $O(n)$, at the cost of $O(n)$ auxiliary space.

CountAndSetLoops For nodes with multiple looping edges, the width of a loop is scaled based on the number of loops. The original implementation finds

Algorithm 1 CountAndSetLayersOld

```

for edgeOuter in edges do
  layers  $\leftarrow$  []
  i  $\leftarrow$  0
  for edgeInner in edges do
    if (edgeInner.head == edgeOuter.head
    and edgeInner.tail == edgeOuter.tail)
    or
    (edgeInner.tail == edgeOuter.head
    and edgeInner.head == edgeOuter.tail) then
      layers[i]  $\leftarrow$  edgeInner
      i  $\leftarrow$  i + 1
    end if
  end for
  edgeOuter.layers  $\leftarrow$  layers
end for

```

Algorithm 2 CountAndSetLayersNew

```

layerMap  $\leftarrow$  new Map
for edge in edges do
  sortedKey  $\leftarrow$  sort([edge.head, edge.tail])
  layerMap.set(sortedKey, layerMap.get(sortedKey) + 1)
end for
for edge in edges do
  sortedKey  $\leftarrow$  sort([edge.head, edge.tail])
  edge.layerCount = layerMap.get(sortedKey)
end for

```

this number similarly to `countAndSetLayers`, adding an array of neighboring edges to each looping edge. The update is also similar to `countAndSetLayers`: whenever a loop is encountered, its tail is used as the key in a hashmap, with the corresponding value being an array, that is appended with the current edge. A second for-loop reads these arrays, adding them as attributes to the edges. Same as `countAndSetLayers`, this results in an improvement from $O(n^2)$ to $O(n)$, at the cost of $O(n)$ auxiliary space.

GetOtherEqualProperty This function is called when creating properties and hiding duplicate connections. It was previously called in every loop iteration on the list of all properties. The function checks if the property provided is equivalent to any other property in the list through a sequential search, an $O(n)$ operation on the number of properties. This is then computed n times - in each loop iteration, leading to $O(n^2)$ time complexity. We found that the computation was only used in one place, which was an *if* statement where it was the second of two boolean clauses connected by an *and* operator:

```
if (propertyWasRerouted && otherEqualProperty) ( ... )
```

Through testing, we found that the first clause was evaluated as false in all test cases. Utilizing short-circuiting, we move the function call into the second clause, thus only calculating the expensive value when the former statement evaluates to true:

```
if (propertyWasRerouted && getOtherEqualProperty()) ( ... )
```

This effectively results in the same worst-case $O(n^2)$ complexity, however, in practice, the best-case is more prevalent leading to an $\Omega(n)$ best-/average-case complexity.

StoreLinksOnNodes We propose removing the nested for-loop comparing each link to each node to find matches, instead only iterating over the array of links and using their existing range and domain object pointers (pointing to the connected nodes) for constant-time lookups. These are then used to append each link to the relevant nodes, pushing the element to the array of connected links on the nodes. This results in n repetitions of two $O(1)$ operations, effectively leading to a time complexity improvement from $O(n \cdot m)$ to $O(n + m)$.

CombineClassesOrProperties Both of these function calls iterate over each *baseObject*, which can be either classes or properties, comparing them to every attribute in order to find a match. Instead of iterating through each attribute, a map of attributes is created and then used to do this lookup in constant time. This results in changing both of these $O(\beta^2)$ functions to $O(\beta)$ with the overhead of creating a map for each function, which requires $O(\beta)$ auxiliary space. Note that these functions are combined to a single function, `combineClassesOrProperties`, meaning their combined performance is measured in Section 5.

FindAutoDefaultDegree To optimize finding the initial degree of collapse, we propose initially iterating over the nodes, saving the length of each *node.links* array as a number in a separate array, *Deg*, then sorting that array from largest to smallest. This works because *Deg* effectively represents, from largest to smallest, the degree of collapsing which would include that node. Thus, without knowing exactly which nodes, we can know that the value of the $i - 1$ 'th place in this array, representing the i 'th largest link count, is the value for the lowest degree of collapsing that would include at least i nodes after the filter is applied. From this we can manage to find the ideal degree of collapsing in constant time, by indexing *Deg* with a number, $d \in \mathbb{N}$, representing the maximum number of rendered nodes. That is, if the limit is 50 nodes, we find the value of index 50 in *Deg*. However, since more than one node may have the same value, we check if the filtered nodes for the given degree of collapsing would include more than the node limit. If this is the case, we increase the degree by one. Thus, the runtime of this method is no longer dependent on the maximum degree of collapse α , resulting in an improvement from $O(\alpha \cdot (n + m))$ to $O(n + m)$.

Caching JSON strings In the original WebVOWL, the JSON string used to load an ontology was always cached, as described in Section 4.1. To avoid unnecessary memory usage, we have placed a 50 MB limitation on each string in the cache. A string larger than this size will not be kept in memory after the ontology is loaded. However, there is still an improvement to be made here, as our current implementation does not remove the string from memory immediately after use, but rather after the ontology is loaded. Solving this issue could yield a significant reduction in memory usage, especially on large ontologies, where the size of the JSON string can exceed 1 GB. This could, in turn, allow even larger ontologies to be loaded, as the memory occupied by the JSON string can be used to load the ontology.

MergeRangesOfEquivalentProperties We begin the method by iterating through all properties and populating two maps with each property’s domain’s and range’s ID respectively. We then continue similarly to the previous implementation by iterating over all properties and for each one finding their equivalents, saving them in an array, and creating an element to represent the merged properties. Following this, we iterate over each equivalent, and use the maps to look up their domain’s and range’s ID to compare it with the range’s ID of the equivalent. If there is no match, we add the range of the equivalent to the list of nodes to hide. This yields an improvement from $O(n^2 \cdot \epsilon)$ to $O(n \cdot \epsilon)$ at the cost of creating the two maps, which requires $O(n)$ auxiliary space.

SubclassFilter When filtering solitary subclasses, the case where a subclass is connected to another subclass is handled by recursively checking connected properties while keeping track of previously visited properties to avoid infinite cycles. In the original implementation, each recursive call iterates over the entire list of properties to find connections, however, this can be avoided by first creating a hashmap mapping a property to its related properties. From this change, the worst-case time complexity is improved from $O(n \cdot (n + m))$ to $O(n^2 + m)$, while using at worst $O(n + m)$ auxiliary memory for the hashmap. Note that in practice the worst case only occurs when all properties are not only subclasses, but connected to all other properties. In the best case the complexity is $\Omega(n + m)$ for both versions, however, in general the updated version uses at most as many operations as the original.

Refactoring to ES6 Rather than using *var* for all variables, the entire codebase is updated to use *let* and *const* where appropriate, in order to aid the JavaScript runtime and garbage collector with optimization. Additionally, as described in Section 4.1, the previous prototype-based classes did not always reference functions in their prototypes. By refactoring all ES5 classes to ES6, the JavaScript runtime can guarantee that objects from the same class inherit the same methods, leading to a significant decrease in memory usage, as shown in table 3. This had the additional benefit of also improving load time, as much of the time spent on garbage collection is no longer necessary.

Searching for graph elements We propose implementing a prefix-based search using a custom trie, as described in Section 3.3. This yields an average-case time complexity of $O(\tau)$, which allows the trie to scale much better with larger ontologies. However, it comes at the cost of additional memory, as described in Section 3.3. Additionally, our approach does not fully follow the semantics of the original implementation, as it performed full string-matching of every element name in the graph, whereas our proposal only matches element names by prefix.

4.3 Extended Search Functionality

After improving the loading performance such that WebVOWL can, within a reasonable time, load ontologies exceeding 100k nodes and edges, we found that the next step was to improve the search functionality. Previously, a text search of an element’s name would highlight the element when clicked, centering the screen on this element. However, any element not shown in the rendered graph due to filtering, e.g. degree of collapse, is unable to be navigated to from a search. To remedy this, we propose an extension which, when searching a hidden element, computes a subgraph G_s of the entire graph G using a breadth-first search (BFS) starting at the searched element v'^9 . The BFS stops traversing G when a certain depth, $k \in \mathbb{N}^+$, is reached¹⁰. G_s then includes all elements reachable from v' in k steps, allowing the user to see nearby connected elements. Using the *Reset* button, the user is then able to return to the big picture view of the graph.

4.4 Earlier alternative plans

While all the previously mentioned improvements were carried out and proved to be helpful, as seen in Section 5, the road toward deciding a course of action had several alternatives which were considered. Initially when presented with the codebase, we considered doing a full rewrite of WebVOWL in Rust, along with implementing a graph database. The reason for this plan was simply that the initial exploration of the codebase found that many segments were in need of rewriting, and before a more thorough investigation of performance bottlenecks, it seemed unlikely that the program could be improved to be as performant as needed without a full rewrite. The plan was to write the Rust code and then use `wasm-pack`¹¹ to convert it to a JavaScript-compatible form through WebAssembly. In Rust, we would use an existing force-directed graph library, extended by us with VOWL elements, in order to utilize the performance advantages inherent to Rust, while maintaining integration with any future JavaScript extension. Another component of this was to potentially circumvent the need for the OWL2VOWL converter to convert files to the VOWL format. It uses the OWL API for parsing which is currently rather slow [20]. Additionally, this might also

⁹ In case the searched element is a property (edge), the domain and range of this edge are the starting nodes.

¹⁰ The depth is currently hard-coded to 2.

¹¹ <https://github.com/rustwasm/wasm-pack>, accessed 6/5/2025

make a full integration of OWL2 and RDF files easier than it would be to adjust the existing WebVOWL and OWL2VOWL code. Having all these considerations in mind, we then looked at the downsides of this approach, the biggest of which was clear. A full rewrite of the codebase would be incredibly time intensive, and the sheer scope of the planned tasks would likely pass beyond the time limit set for our bachelor project. Because of this, along with the finding that there was in fact a viable path towards the desired performance in working with the existing code, we decided to focus on that. However, if further performance optimizations are needed, such as those discussed in Section 6, we find that a full rewrite may be the best course of action.

5 Results

The effects of the changes described in Section 4.2 are given in table 2, describing results both in terms of time complexity, and in actual runtime on select inputs. Performance is measured on the ontologies FOAF¹², ENVO¹³, and on the knowledge graph YAGO¹⁴. The provided node and edge count is measured by WebVOWL. Comparisons were completed using the Firefox Profiler to measure the same operation, with and without the described improvement, and each measurement has been repeated 3 times and averaged. The speedup is then given by the time difference, $\frac{\text{original time}}{\text{new time}} = \text{speedup}$. The runtime difference of the completed version with all changes is given, and the difference in peak memory usage is shown in table 3.

¹² Friend of a Friend ($n = 70$, $m = 50$) <http://xmlns.com/foaf/spec/20140114.html>.

¹³ The Environment Ontology [5/2/2025] ($n = 12387$, $m = 7038$) [4] <https://bioportal.bioontology.org/ontologies/ENVO>.

¹⁴ YAGO, tiny version [4.5] ($n = 166425$, $m = 132882$) [27] <https://yago-knowledge.org/data/yago4.5/>.

Table 2. Overview of time complexity improvements ($n = \#edges$, $m = \#nodes$). Load times were measured using the Firefox Profiler on multiple inputs.

Improvement	Time complexity (original $\rightarrow$ new)	Load time improvement (input: original/new)
CountAndSetLayers	$O(n^2) \rightarrow O(n)$	FOAF: $1.5s/0.6s = 2.5$ ENVO: $631.7s/594.9s = 1.06$ YAGO: DNC*
CountAndSetLoops	$O(n^2) \rightarrow O(n)$	FOAF: $1.5s/1.6s = 0.94$ ENVO: $631.7s/407.2s = 1.55$ YAGO: DNC
GetOther- EqualProperty	$O(n^2) \rightarrow O(n^2)$ $\Omega(n^2) \rightarrow \Omega(n)^\dagger$	FOAF: $1.5s/2.0s = 0.75$ ENVO: $631.7s/564.0s = 1.12$ YAGO: DNC
StoreLinksOnNodes	$O(n \cdot m) \rightarrow O(n + m)$	FOAF: $1.5s/2.0s = 0.75$ ENVO: $631.7s/494.5s = 1.28$ YAGO: DNC
CombineClasses- OrProperties	$O(\beta^2) \rightarrow O(\beta)$	FOAF: $1.5s/2.4s = 0.63$ ENVO: $631.7s/546.4s = 1.16$ YAGO: DNC
MergeRangesOf- EquivalentProperties	$O(n^2 \cdot \epsilon) \rightarrow O(n \cdot \epsilon)$	FOAF: $1.5s/0.9s = 1.67$ ENVO: $631.7s/282.7s = 2.23$ YAGO: DNC
SubclassFilter [‡]	$O(n \cdot (n + m)) \rightarrow O(n^2 + m)$	FOAF: $0.2s/0.2s = 1$ ENVO: $63.8s/0.6s = 106.33$ YAGO: DNC/7.6s
Search [‡]	$O(k \cdot t(f - t + 1)) \rightarrow O(\tau)^\S$	FOAF: $7.33ms/5.66ms = 1.3$ ENVO: $51.33ms/23.66ms = 2.17$ YAGO: DNC/88.67ms
All changes [¶]		FOAF: $1.5s/0.23s = 6.52$ ENVO: $631.7s/1.23s = 513.58$ YAGO: DNC/18.7s

* DNC (Did Not Complete) used when the loading time exceeded 20 minutes.

[†] Best-/average-case.[‡] Result is feature runtime, not overall loading time.[§] Average-case.[¶] Includes additional changes described in sections 4.1, 4.2.

Table 3. Overview of peak memory usage. Memory usage was measured using the Firefox Profiler on multiple inputs.

Input	Peak memory usage (original/all changes)	Reduced by
FOAF	22.8MB/15.1MB	34%
ENVO	524MB/227.33MB	57%
YAGO	DNC/3.95GB	

6 Discussion

These results shown in Section 5 have proven to be quite significant, and are on a scale that does not just make WebVOWL more convenient to use, but also makes it usable with much larger ontologies than previously possible. In light of the 2018 survey by Dudáš et al. [8], we believe that this new and improved WebVOWL may be only one of two ontology visualization tools based on force-directed graphs which are capable of loading ontologies exceeding 100k nodes and edges¹⁵.

What is still a limiting factor for loading even larger ontologies than those in our benchmarks is simply memory usage. Loading YAGO, tiny version, used almost 4 GB of memory, and as memory used scales linearly with the size of the ontology, some users would not have the required amount of RAM needed to load something with, for example, a million nodes and edges. If anyone was to further improve on WebVOWL, improving memory efficiency may be a key place to start. Section 4.2 describes an unsolved issue regarding memory efficiency. Other improvements still missing in the new version would be extending support to more diverse formats, e.g. RDF, as well as improving performance of the OWL2VOWL converter, which in its current state can be a lot slower than loading the file itself in WebVOWL, especially on large inputs such as YAGO.

With these considerations in mind, we believe that WebVOWL can become a significant resource in the development, exploration, and presentation of ontologies.

7 Conclusion

In this paper, we present the results of our efforts to reduce loading time and memory usage of WebVOWL, in addition to how we have extended the search functionality with a new feature. We conducted benchmarks using real data of different sizes, each size increasing node and edge count by two orders of magnitude from the previous.

¹⁵ The other being TGViz. However, it has, as of this writing, not been updated since 2005.

The performance improvements to loading times have proven to be quite significant in these benchmarks; an ontology which before took 631.7 seconds to load can now be loaded in 1.23 seconds, i.e. 514 times faster. Additionally, memory usage was reduced by 34-57%, depending on input. Our improved search allows the user to search for any element in the graph and if the searched element is not currently rendered on screen, a subgraph is created such that the searched element and its nearby connected elements are shown, replacing the previously rendered graph. Our benchmarks also show that searching is 70% faster on average¹⁶.

Further optimization is still possible and for future work, the remaining bottlenecks are memory usage, graph rendering, and loading ontologies from OWL format.

The code for the improved WebVOWL is available online¹⁷. We hope that future use of the tool in relevant fields may be positively affected by our findings.

References

1. Alani, H.: Tgviz, <https://protegewiki.stanford.edu/wiki/TGViz>
2. Berners-Lee, T., Hendler, J., Lassila, O.: The semantic web. *Scientific American* **284**(5), 34–43 (2001), <http://www.jstor.org/stable/26059207>
3. for Biomedical Informatics Research (BMIR), S.C.: Protégé home-page, <https://protege.stanford.edu/>
4. Buttigieg, P.L., Pafilis, E., Lewis, S.E., Schildhauer, M.P., Walls, R.L., Mungall, C.J.: The environment ontology in 2016: bridging domains with increased scope, semantic density, and interoperability. *Journal of Biomedical Semantics* **7**(57) (2016). <https://doi.org/10.1186/s13326-016-0097-6>
5. contributors, M.: let (03 2025), <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/let>
6. contributors, M.: Memory management (03 2025), https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Memory_management
7. Cormen, T., Leiserson, C., Rivest, R., Stein, C.: Introduction to Algorithms, fourth edition. MIT Press (2022), <https://books.google.dk/books?id=HOJyzgEACAAJ>
8. Dudáš, M., Lohmann, S., Svátek, V., Pavlov, D.: Ontology visualization methods and tools: a survey of the state of the art. *The Knowledge Engineering Review* **33** (07 2018). <https://doi.org/10.1017/S0269888918000073>
9. Ecma International: ECMAScript® 2015 Language Specification, 6 edn., https://ecma-international.org/wp-content/uploads/ECMA-262_6th_edition_june_2015.pdf
10. Ecma International: ECMAScript® 2024 Language Specification, 15 edn., https://ecma-international.org/wp-content/uploads/ECMA-262_15th_edition_june_2024.pdf
11. Group, W.O.W.: Owl 2 web ontology language (12 2012), <https://www.w3.org/TR/owl2-overview/>
12. Haag, F., Lohmann, S., Negru, S., Ertl, T.: Ontovibe: An ontology visualization benchmark **1299**, 14–27 (01 2014), <https://ceur-ws.org/Vol-1299/paper2.pdf>

¹⁶ Note that creating the subgraph of a searched element is not included in this.

¹⁷ <https://github.com/WebVOWL/WebVOWL-Legacy>

13. Hadzic, M., Wongthongtham, P., Dillon, T., Chang, E.: Introduction to Ontology, pp. 37–60. Springer Berlin Heidelberg, Berlin, Heidelberg (2009). https://doi.org/10.1007/978-3-642-01904-3_3
14. Heflin, J.: An introduction to the owl web ontology language. Lehigh University. National Science Foundation (NSF) **7** (2007)
15. Janowicz, K., Schlobach, S., Lohmann, S., Negru, S., Haag, F., Ertl, T.: Visualizing ontologies with vowl. *Semantic Web* **7**(4), 399–419 (2016). <https://doi.org/10.3233/SW-150200>
16. Kantor, I.: Garbage collection (10 2022), <https://javascript.info/garbage-collection>
17. Knuth, D.: The Art of Computer Programming: Sorting and Searching, Volume 3. Pearson Education (1998), <https://books.google.dk/books?id=cYULBAAAQBAJ>
18. Lohmann, S., Link, V., Marbach, E., Negru, S.: Webvowl: Web-based visualization of ontologies. p. 154–158. Springer-Verlag, Berlin, Heidelberg (2014). https://doi.org/10.1007/978-3-319-17966-7_21
19. Lohmann, S., Negru, S., Bold, D.: The protégévowl plugin: Ontology visualization for everyone. In: Presutti, V., Blomqvist, E., Troncy, R., Sack, H., Papadakis, I., Tordai, A. (eds.) *The Semantic Web: ESWC 2014 Satellite Events*. pp. 395–400. Springer International Publishing, Cham (2014)
20. Lord, P., Gehrke, B., Larralde, M., Hastings, J., De Bortoli, F., Overton, J.A., Balhoff, J.P., Warrender, J.: Horned-OWL: Flying Further and Faster with Ontologies. *Transactions on Graph Data and Knowledge* **2**(2), 9:1–9:14 (2024). <https://doi.org/10.4230/TGDK.2.2.9>
21. Musen, M.A.: The protégé project: a look back and a look forward. *AI Matters* **1**(4), 4–12 (Jun 2015). <https://doi.org/10.1145/2757001.2757003>
22. Musen, M.A., Fagan, L.M., Combs, D.M., Shortliffe, E.H.: Use of a domain model to drive an interactive knowledge-editing tool. *International Journal of Man-Machine Studies* **26**(1), 105–121 (1987). [https://doi.org/10.1016/S0020-7373\(87\)80039-1](https://doi.org/10.1016/S0020-7373(87)80039-1)
23. Niles, I., Pease, A.: Towards a standard upper ontology. In: *Proceedings of the International Conference on Formal Ontology in Information Systems - Volume 2001*. p. 2–9. FOIS '01, Association for Computing Machinery, New York, NY, USA (2001). <https://doi.org/10.1145/505168.505170>
24. Pretzmann, F., Nielsen, K., Rennekamp, M., Niemi, N., Sagi, T.: Making webvowl great again. In: *The Semantic Web: ESWC 2025 Satellite Events, Portoroz, Slovenia, June 1 - 5, 2025, Proceedings*. Springer Nature, United States (Apr 2025)
25. Shneiderman, B.: The eyes have it: a task by data type taxonomy for information visualizations. In: *Proceedings 1996 IEEE Symposium on Visual Languages*. pp. 336–343 (1996). <https://doi.org/10.1109/VL.1996.545307>
26. Smith, M.K., Welty, C., McGuinness, D.L.: Owl web ontology language guide (2 2004), <https://www.w3.org/TR/owl-guide/>
27. Suchanek, F.M., Alam, M., Bonald, T., Chen, L., Paris, P.H., Soria, J.: Yago 4.5: A large and clean knowledge base with a rich taxonomy. In: *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. p. 131–140. SIGIR '24, Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3626772.3657876>